

Monte Carlo Stock Forecasting - Case Studies

Introduction

Quantitative financial analysis aims to model and interpret market behaviour through statistical and mathematical methods. One of the central challenges within this field is the estimation of future stock price behaviour under uncertainty. While financial markets are inherently unpredictable, historical price movements can still provide valuable information about volatility, trends, and probabilistic future outcomes. As a result, many forecasting approaches attempt to use historical market behaviour as a foundation for estimating future price ranges rather than exact deterministic predictions.

This project investigates the effectiveness and limitations of a Monte Carlo based stock forecasting framework across equities with differing market characteristics and volatility regimes. To evaluate model behaviour under a variety of conditions, the analysis focuses on six stocks: SPY as a broad market benchmark, KO as a defensive low-volatility stock, MSFT as a mature technology company, TSLA as a high-volatility growth stock, and META and NVDA as momentum-driven technology equities with significant trend behaviour. Through these case studies, the project explores where probabilistic forecasting performs well, where it struggles, and how differing market structures influence model accuracy and uncertainty estimation. The broader goal is not the creation of a perfect predictive system, but rather the identification of model strengths, structural weaknesses, and potential avenues for future improvement and development.

Methodology

Monte Carlo Forecasting Framework

Monte Carlo simulations are a probabilistic forecasting technique used to model systems with inherent uncertainty. Rather than attempting to predict a single deterministic future outcome, the method generates a large number of possible future price paths based on historical statistical behaviour. By analysing the distribution of these simulated outcomes, it becomes possible to estimate expected price behaviour, uncertainty ranges, and the likelihood of extreme events.

In this project, stock prices are simulated using 1000 Monte Carlo paths over a forecasting horizon of 126 trading days, corresponding approximately to half a trading year. The simulation framework is based on historical percentage price changes, allowing future stock behaviour to emerge from previously observed market dynamics.

```
In [1]: #Adding all required imports
import sys
import os
sys.path.append(os.path.abspath("."))
from datetime import date
from src.stats import gaussian_params
from src.data import download_stock_data, clean_stock_data
from src.simulation import montecarlo_iteration
from src.evaluation import forecast_dispersion_width, jump_comparison, maximum_absolute_percentage_error, rolling_root_
from src.visualization import histogram, plot_results
import matplotlib.pyplot as plt
import numpy as np
import plotly.io as pio

pio.renderers.default = "vscode+notebook"

#setting default parameters used for the case studies and a seed to make it repeatable
np.random.seed(42)

#the specific stock name will be defined in the individual case studies, as we want to test different stocks.
stock_name = ["SPY", "KO", "MSFT", "TSLA", "META", "NVDA"]

#We are not defining the allow jumps as this may be set to True or False depending on the case study
training_start_date = date(2015, 1, 1)
evaluation_start_date = date(2023, 1, 1)
evaluation_end_date = date.today() #This will automatically be cut to the montecarlo days if it exceeds it in the pl
montecarlo_days = 126
montecarlo_iterations = 1000
sigma_multiplier = 2.5
confidence_level = 0.95
```

Data Acquisition and Cleaning

Historical market data is obtained directly from Yahoo Finance using the yfinance library. For each stock, historical price data from 01/01/2015 to 01/01/2023 is used as the model training period. The subsequent 126 trading days are then reserved as evaluation

data, allowing simulated forecasts to be directly compared against real market behaviour.

Before being used within the simulation, the downloaded data is cleaned to remove missing values and ensure numerical consistency across all calculations.

```
In [2]: #downloading the data (and cleaning it) to be added to a dictionary for all stocks.

training_stock_data = {}
evaluation_stock_data = {}
for stock in stock_name:
    stock_data_train = download_stock_data(stock, training_start_date, evaluation_start_date)
    training_stock_data[stock] = clean_stock_data(stock_data_train)

    stock_data_eva = download_stock_data(stock, evaluation_start_date, evaluation_end_date)
    evaluation_stock_data[stock] = clean_stock_data(stock_data_eva)
```

```
[*****100%*****] 1 of 1 completed
[*****100%*****] 1 of 1 completed
[*****100%*****] 1 of 1 completed
[*****100%*****] 1 of 1 completed
[*****100%*****] 1 of 1 completed
[*****100%*****] 1 of 1 completed
[*****100%*****] 1 of 1 completed
[*****100%*****] 1 of 1 completed
[*****100%*****] 1 of 1 completed
[*****100%*****] 1 of 1 completed
[*****100%*****] 1 of 1 completed
[*****100%*****] 1 of 1 completed
[*****100%*****] 1 of 1 completed
[*****100%*****] 1 of 1 completed
[*****100%*****] 1 of 1 completed
```

The montecarlo simulation for each stock is then generated using the cleaned data with a gaussian distribution model.

```
In [3]: #Storing all monetcarlo simulations in a dictionary to use later (and make it look a bit less crowded within teh case

# Structure:
# montecarlo_data[stock_name][0] -> no jumps
# montecarlo_data[stock_name][1] -> jumps enabled
#

montecarlo_data = {}

for name in stock_name:

    montecarlo_data[name] = [
        montecarlo_iteration(
            data = training_stock_data[name],
            bin_width = 0.1/100,
            sigma_multiplier = sigma_multiplier ,
            days = montecarlo_days,
            iterations = montecarlo_iterations,
            allow_jumps=False
        ),

        montecarlo_iteration(
            data = training_stock_data[name],
            bin_width = 0.1/100,
            sigma_multiplier = sigma_multiplier ,
            days = montecarlo_days,
            iterations = montecarlo_iterations,
            allow_jumps=True
        )
    ]
```

We justify the gaussian model choice and the standard deviation multiplier for the jump cutoff below.

Historical Return Distribution Modeling

Rather than simulating raw stock prices directly, the model operates on historical percentage price changes. This normalisation allows stocks with very different absolute price scales to be analysed within the same probabilistic framework.

To estimate the underlying behaviour of these returns, the historical percentage changes are visualised as a histogram and approximated using a Gaussian probability distribution. The Gaussian distribution used throughout the simulation framework is given by:

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

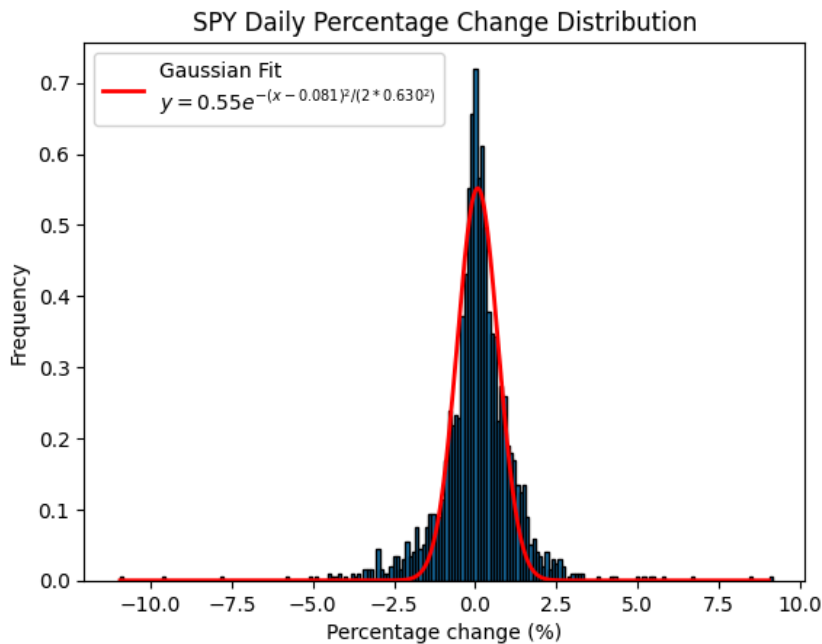
where (\mu) represents the mean return and (\sigma) the standard deviation of the historical return distribution. This produces a smooth probabilistic approximation of expected daily market movement centred around the historical mean return.

While financial markets are not perfectly Gaussian in nature, the approximation provides a computationally efficient baseline model which reproduces the majority of observed daily return behaviour reasonably well.

```
In [4]: #We show an example here of the histogram used, this will be done internally for each montecarlo simulation.
#The example here is for SPY where .pct_change() calculates the percentage change between adjacent data points (in th
```

```
training_stock_data["SPY"]["pct_change"] = training_stock_data["SPY"]["Close"].pct_change()

histogram(
    training_stock_data["SPY"]["pct_change"].dropna() * 100, # to get as a %
    bin_width = 0.1, #some small bin size to illustrate it nicely
    gaussian_fit = True,
    title = "SPY Daily Percentage Change Distribution",
    xlabel = "Percentage change (%)",
    ylabel = "Frequency",
    show_graph = True,
    save_graph=False)
```



Jump Process Extension

Financial markets occasionally experience rare extreme events which cannot be accurately represented by a standard Gaussian distribution alone. To partially account for these events, the simulation framework optionally incorporates a jump process.

Jumps are defined as historical return events exceeding 2.5 standard deviations (at least for these case studies, the value can be adjusted by the user) from the mean return distribution. During simulation, these rare events can be sampled separately and introduced into simulated price paths, allowing the model to better reproduce sudden market shocks and periods of elevated volatility.

The inclusion of jumps significantly improves the realism of the simulated forecasts, particularly for highly volatile equities.

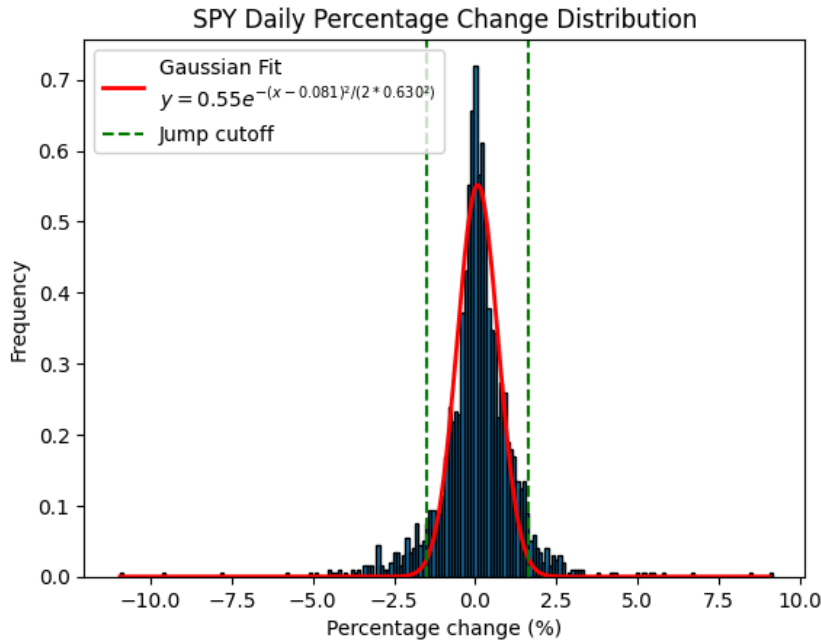
```
In [5]: histogram(
    training_stock_data["SPY"]["pct_change"].dropna() * 100,
    bin_width = 0.1, #some small bin size to illustrate it nicely
    gaussian_fit = True,
    title = "SPY Daily Percentage Change Distribution",
    xlabel = "Percentage change (%)",
    ylabel = "Frequency",
    show_graph = False,
    save_graph = False)

#adding the cutoff lines
params = gaussian_params( training_stock_data["SPY"]["pct_change"].dropna() * 100, bin_width = 0.1)
mean = params[1]
sd = params[2]

plt.axvline(x=(mean + sigma_multiplier*sd), color='green', linestyle='dashed')
plt.axvline(x=(mean - sigma_multiplier*sd), color='green', linestyle='dashed', label = 'Jump cutoff')

plt.legend()

plt.show()
```



Forecast Evaluation Metrics

To evaluate simulation quality, forecast outputs are compared directly against real stock price behaviour during the evaluation period. Several statistical metrics are used throughout the analysis.

The Root Mean Squared Error (RMSE) measures the average magnitude of forecast error while penalising larger deviations more heavily. RMSE is defined as:

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

where (y_i) represents the real stock price and ($\hat{y}_i$) the predicted stock price at each timestep.

The Mean Absolute Percentage Error (MAPE) measures forecast error relative to the true stock price, allowing comparisons across equities with different price scales:

$$MAPE = \frac{100}{n} \sum_{i=1}^n \left| \frac{y_i - \hat{y}_i}{y_i} \right|$$

Prediction interval coverage is additionally used to determine how frequently the real stock price remains inside the simulated confidence interval estimates:

$$Coverage = \frac{\text{Number of observations inside interval}}{\text{Total observations}} \times 100$$

The rate at which this confidence band spreads is also calculated, if the value is larger it means the stock can be expected to be historically less stable and hence the confidence interval cone grows more rapidly.

In addition to these statistical measures, jump statistics are compared between simulated and real market behaviour to evaluate how effectively the jump-process extension reproduces rare extreme-event dynamics.

Model Assumptions and Limitations

Several simplifying assumptions are made within the forecasting framework.

- Historical market behaviour is assumed to contain information relevant to future price behaviour.
- Daily returns are approximated using a Gaussian distribution.
- Trading days are treated as statistically independent within the simulation process.
- The model does not explicitly account for macroeconomic events or market regime shifts.
- Cross-market dependencies between equities are not included.
- Sudden structural changes in company fundamentals cannot be predicted by the model.

These assumptions allow the framework to remain computationally efficient and interpretable, while also highlighting several potential areas for future model development and improvement.

Case Studies

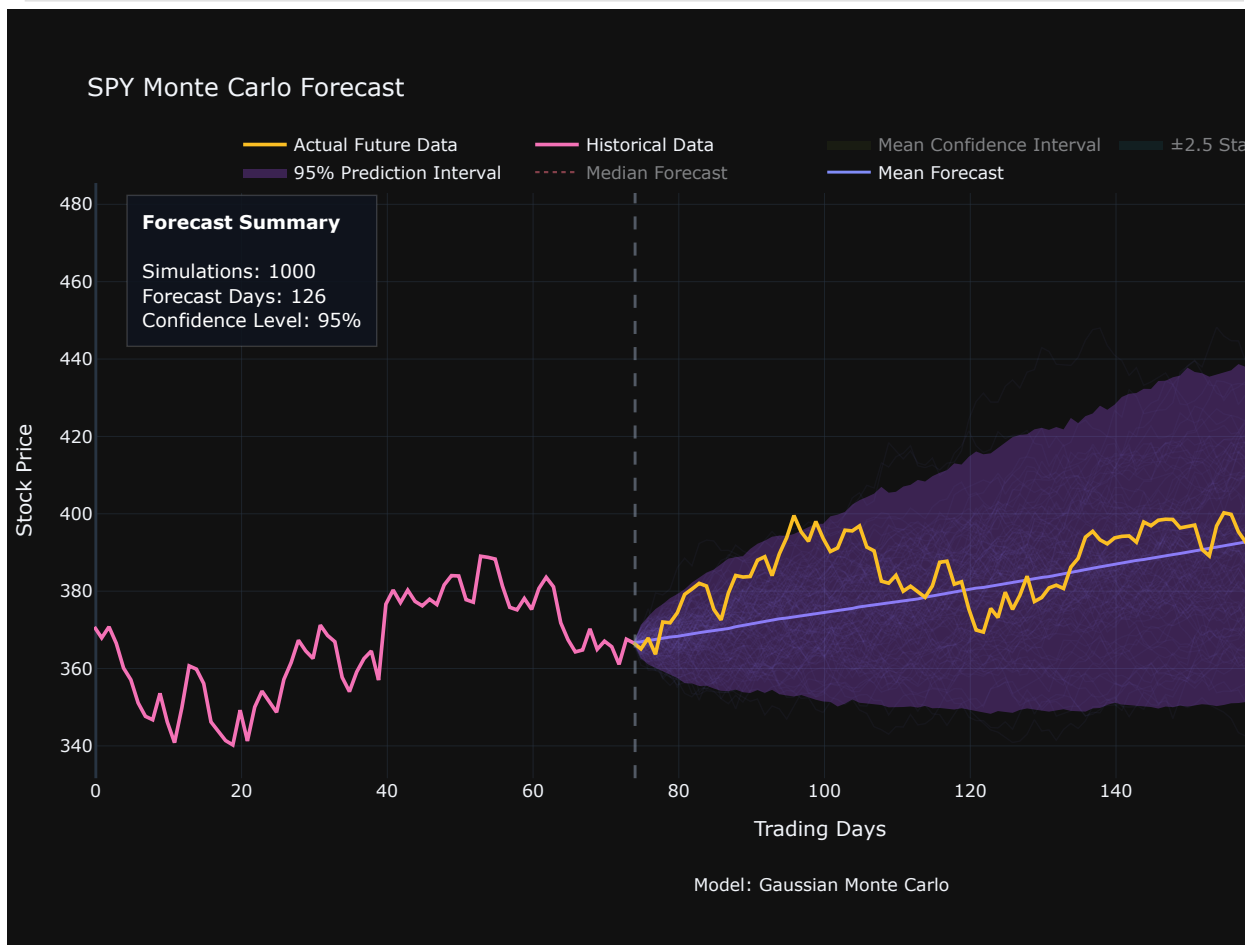
SPY (SPDR S&P 500 ETF Trust)

Broad market benchmark representing the overall U.S. equity market. Included to evaluate how the model performs on diversified, lower-volatility aggregate market behaviour.

Montecarlo Graphs and Evaluation

```
In [6]: plot_results(
    simulations = montecarlo_data[stock_name[0]][0], #[0] as we are showing the jump free graph first
    confidence_level = confidence_level,
    sd_multiplier=sigma_multiplier,
    stock_name = stock_name[0],
    historical_data = training_stock_data[stock_name[0]]["Close"],
    actual_data = evaluation_stock_data[stock_name[0]]["Close"].iloc[0:montecarlo_days-1],
    allow_jumps = False,
    show_figure = True
)
#the remaining parameters can be the default since plotly allows for interactive graphs

#the evaluation parameters to be shown.
show_evaluation(
    simulations = montecarlo_data[stock_name[0]][0],
    actual_data = evaluation_stock_data[stock_name[0]]["Close"],
    training_data = training_stock_data[stock_name[0]]["Close"],
    stock_name = stock_name[0],
    jumps_enabled=False,
    confidence_level=confidence_level
)
```



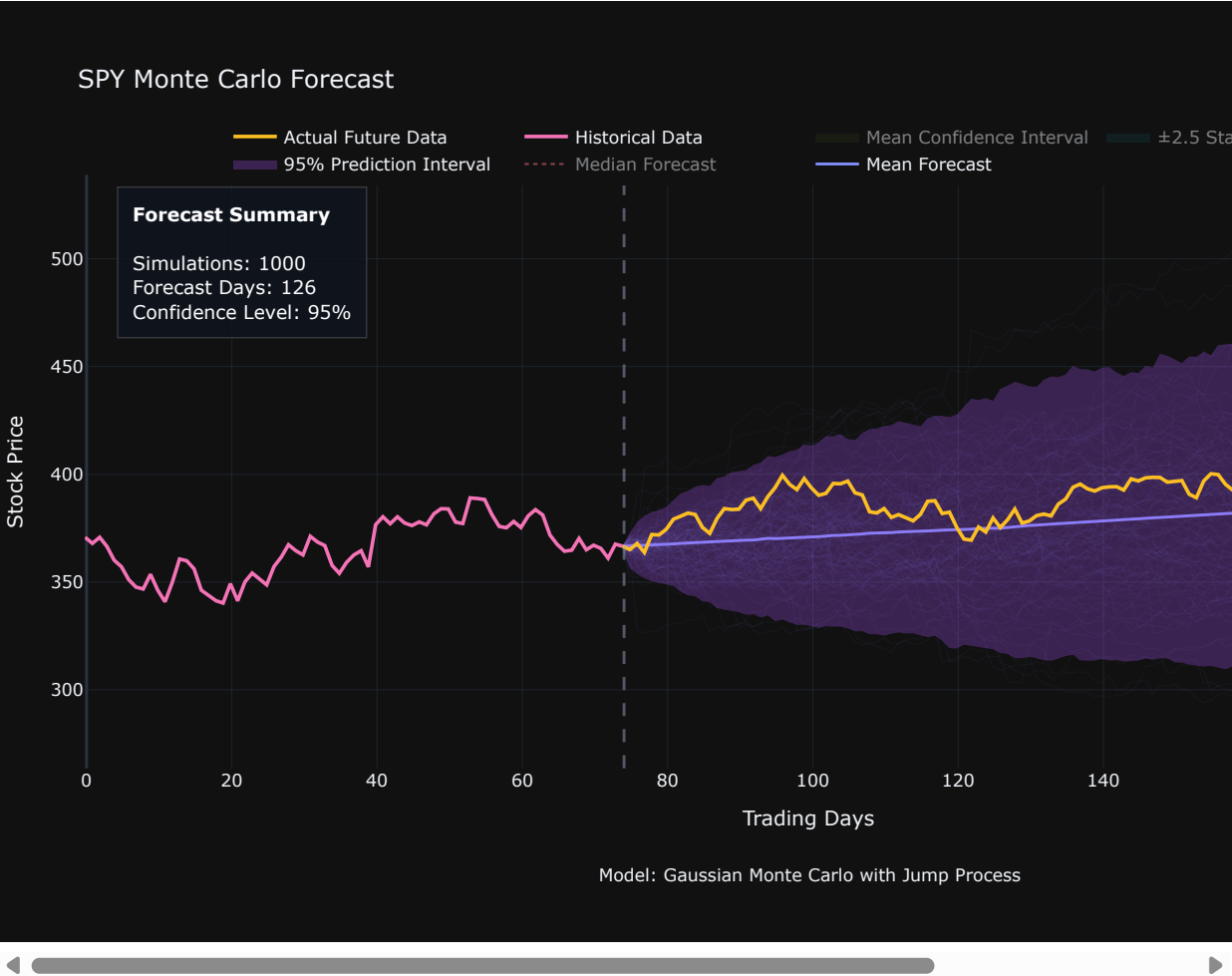
SPY Standard Model			
RMSE	11.5049	MAPE (%)	2.3478
Coverage (%)	96.03	Dispersion	0.926279
	Real	Simulation	
Positive Jumps	0	0.00	
Negative Jumps	0	0.00	
Avg Positive Jump	0.0000	0.0000	
Avg Negative Jump	0.0000	0.0000	

The standard Gaussian Monte Carlo model produced the strongest overall performance for SPY during the validation period. Both the RMSE and MAPE values remained relatively low, indicating that the simulated price trajectories stayed close to the realised market behaviour throughout the forecast horizon. In addition, the prediction interval coverage of 96.03% was very close to the target 95% confidence level, suggesting that the model produced well-calibrated uncertainty estimates. No significant jump events were detected in the real market data, which is consistent with the smoother and more stable behaviour typically associated with broad market stocks SPY.

```
In [7]: plot_results(
    simulations = montecarlo_data[stock_name[0]][1], #[0] as we are showing the jump free graph first
    confidence_level = confidence_level,
    sd_multiplier=sigma_multiplier,
    stock_name = stock_name[0],
    historical_data = training_stock_data[stock_name[0]]["Close"],
    actual_data = evaluation_stock_data[stock_name[0]]["Close"].iloc[0:montecarlo_days-1],
    allow_jumps = True
).show()

#the remaining parameters can be the default since plotly allows for interactive graphs

#the evaluation parameters to be shown.
show_evaluation(
    simulations = montecarlo_data[stock_name[0]][1],
    actual_data = evaluation_stock_data[stock_name[0]]["Close"],
    training_data = training_stock_data[stock_name[0]]["Close"],
    stock_name = stock_name[0],
    jumps_enabled=True,
    confidence_level=confidence_level
)
```



=====			
SPY With Jumps			
=====			
RMSE	18.6852	MAPE (%)	4.0427
Coverage (%)	99.21	Dispersion	1.558903

	Real	Simulation	
Positive Jumps	0	2.31	
Negative Jumps	0	3.16	
Avg Positive Jump	0.0000	0.0402	
Avg Negative Jump	0.0000	-0.0372	

The jump diffusion model introduced substantially greater forecast volatility and uncertainty into the simulation paths. Although interval coverage increased slightly to 99.21%, this came at the cost of noticeably higher RMSE, MAPE and dispersion values, indicating a reduction in overall predictive accuracy. The model also generated multiple positive and negative jump events despite no comparable jumps occurring in the real validation data. This suggests that the jump diffusion extension overestimated the likelihood of extreme market movements during the forecast period and therefore produced unnecessarily volatile simulation behaviour for SPY. Graphically this can be seen by a confidence interval cone that disperses quicker than in the no jump model.

Overall, the no jump framework appears to provide the most appropriate representation of SPY within this case study. This result is financially intuitive, as SPY represents a highly diversified market index where extreme movements from individual companies are naturally averaged out across the broader market. As a consequence, aggregate market behaviour tends to evolve more smoothly and exhibit fewer abrupt jumps outside of major macroeconomic or regime-shift events. The poorer performance of the jump diffusion model is therefore not unexpected. It is also worth noting that the jump model may have overestimated jump frequency due to calibration based on historical jump occurrence, causing the simulation to introduce more extreme movements than were actually present during the validation period.

KO (Coca-Cola)

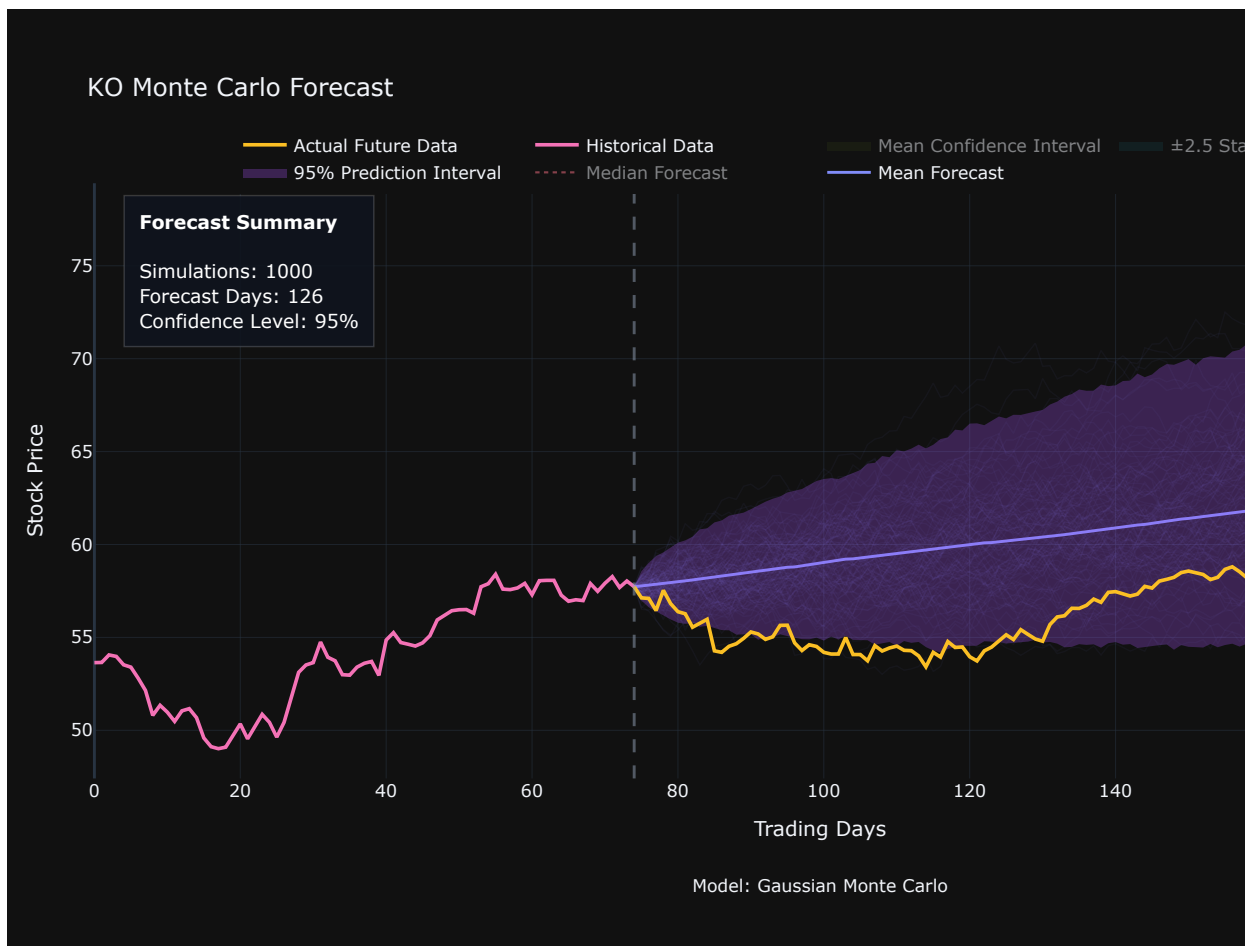
Defensive consumer staples company known for relatively stable long-term price behaviour. Included as an example of a lower-volatility equity with mature market dynamics.

Montecarlo Graphs and Evaluation

```
In [8]: plot_results(
    simulations = montecarlo_data[stock_name[1]][0], #[0] as we are showing the jump free graph first
    confidence_level = confidence_level,
    sd_multiplier=sigma_multiplier,
    stock_name = stock_name[1],
    historical_data = training_stock_data[stock_name[1]]["Close"],
    actual_data = evaluation_stock_data[stock_name[1]]["Close"].iloc[0:montecarlo_days-1],
    allow_jumps = False
).show()

#the remaining parameters can be the default since plotly allows for interactive graphs

#the evaluation parameters to be shown.
show_evaluation(
    simulations = montecarlo_data[stock_name[1]][0],
    actual_data = evaluation_stock_data[stock_name[1]]["Close"],
    training_data = training_stock_data[stock_name[1]]["Close"],
    stock_name = stock_name[1],
    jumps_enabled=False,
    confidence_level=confidence_level
)
```



KO | Standard Model

RMSE	5.0978	MAPE (%)	8.5453
Coverage (%)	73.02	Dispersion	0.157478

	Real	Simulation
Positive Jumps	0	0.07
Negative Jumps	1	0.05
Avg Positive Jump	0.0000	0.0250
Avg Negative Jump	-0.0303	-0.0240

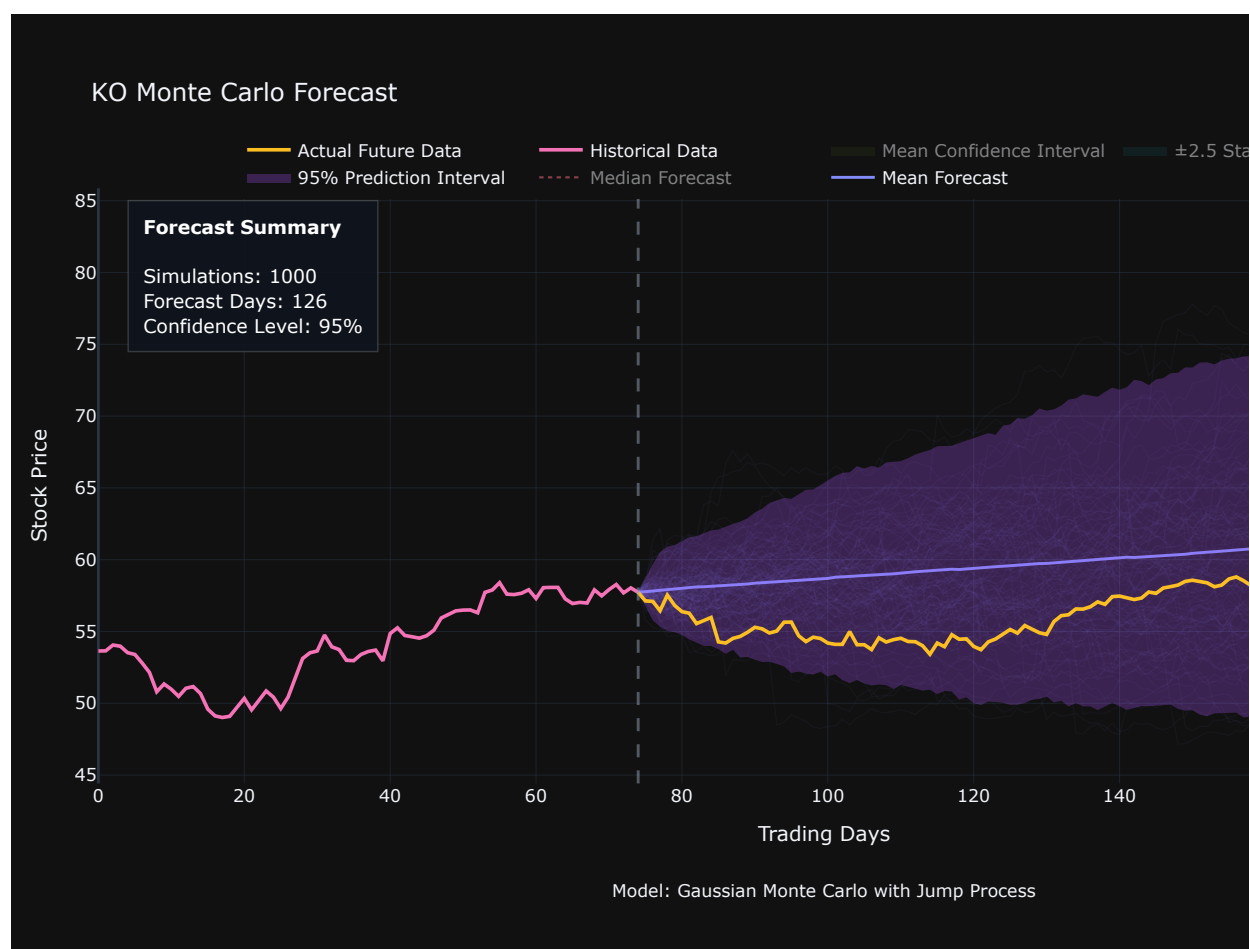
Our standard model produced reasonable forecasts for Coca-Cola, although performance was noticeably weaker than in the SPY case study. While forecast dispersion remained relatively low, reflecting the stable and defensive nature of the stock, the prediction interval coverage fell to 73.02%, well below the target confidence level. This suggests that the standard framework underestimated the probability of larger market movements during the validation period.

The presence of a real negative jump event in the observed data was sufficient to push real prices outside the narrower prediction intervals generated by the model. Despite this, the relatively modest dispersion values remain financially intuitive, as Coca-Cola historically exhibits lower volatility and fewer extreme movements compared to more growth-oriented equities.

```
In [9]: plot_results(
    simulations = montecarlo_data[stock_name[1]][1],
    confidence_level = confidence_level,
    sd_multiplier=sigma_multiplier,
    stock_name = stock_name[1],
    historical_data = training_stock_data[stock_name[1]]["Close"],
    actual_data = evaluation_stock_data[stock_name[1]]["Close"].iloc[0:montecarlo_days-1],
    allow_jumps = True
).show()

#the remaining parameters can be the default since plotly allows for interactive graphs

#the evaluation parameters to be shown.
show_evaluation(
    simulations = montecarlo_data[stock_name[1]][1],
    actual_data = evaluation_stock_data[stock_name[1]]["Close"],
    training_data = training_stock_data[stock_name[1]]["Close"],
    stock_name = stock_name[1],
    jumps_enabled=True,
    confidence_level=confidence_level
)
```

=====			
KO With Jumps			
=====			
RMSE	4.3437	MAPE (%)	7.2525
Coverage (%)	99.21	Dispersion	0.243570

	Real	Simulation	
Positive Jumps	0	1.97	
Negative Jumps	1	2.23	
Avg Positive Jump	0.0000	0.0417	
Avg Negative Jump	-0.0303	-0.0438	

Introducing jump diffusion dynamics improved overall predictive performance across several key metrics. Both RMSE and MAPE decreased, indicating that the model was better able to capture the realised price trajectory once occasional discontinuous movements were incorporated into the simulation process. Prediction interval coverage also increased substantially to 99.21%, suggesting that the jump diffusion framework provided a more robust representation of uncertainty during the forecast horizon. However, this improvement came alongside a moderate increase in forecast dispersion and a clear tendency to overestimate jump frequency.

Although only a single negative jump occurred in the real validation data, the model generated multiple positive and negative jumps on average across the simulations. This indicates that the jump process may still be somewhat overfitted to historical jump behaviour despite the numerical parameter optimisation performed during model calibration.

MSFT (Microsoft)

Large-cap technology company with comparatively stable growth and strong institutional presence. Included to study how the model performs on mature technology equities.

Montecarlo Graphs and Evaluation

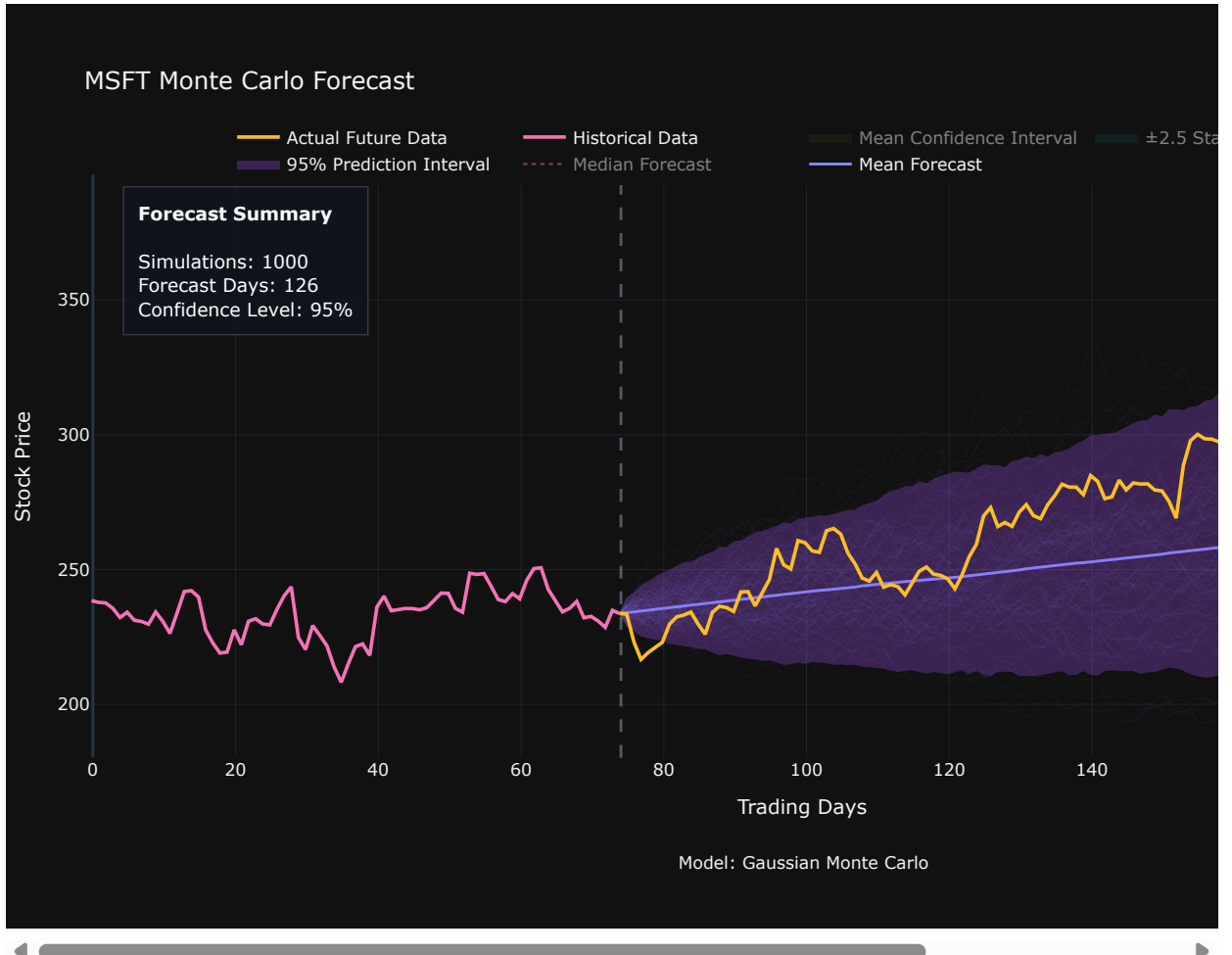
```
In [10]: plot_results(
    simulations = montecarlo_data[stock_name[2]][0],
    confidence_level = confidence_level,
    sd_multiplier=sigma_multiplier,
    stock_name = stock_name[2],
    historical_data = training_stock_data[stock_name[2]]["Close"],
    actual_data = evaluation_stock_data[stock_name[2]]["Close"].iloc[0:montecarlo_days-1],
    allow_jumps = False
```

```

).show()
#the remaining parameters can be the default since plotly allows for interactive graphs

#the evaluation parameters to be shown.
show_evaluation(
    simulations = montecarlo_data[stock_name[2]][0],
    actual_data = evaluation_stock_data[stock_name[2]]["Close"],
    training_data = training_stock_data[stock_name[2]]["Close"],
    stock_name = stock_name[2],
    jumps_enabled=False,
    confidence_level=confidence_level
)

```



MSFT Standard Model			
RMSE	35.8587	MAPE (%)	9.5146
Coverage (%)	91.27	Dispersion	1.044892
	Real	Simulation	
Positive Jumps	5	0.07	
Negative Jumps	1	0.05	
Avg Positive Jump	0.0481	0.0378	
Avg Negative Jump	-0.0437	-0.0357	

Microsoft exhibited substantially more jump behaviour during the validation period than the previous case studies, with multiple significant positive jump events occurring. Our standard model still produced relatively strong overall performance, achieving lower RMSE and MAPE values than the jump diffusion extension. However, the model underestimated the frequency of discontinuous movements, resulting in slightly weaker prediction interval coverage and comparatively narrow forecast dispersion. The nearly negligible jump counts generated by the standard model further highlight its inability to fully capture the heavier-tailed behaviour observed within the real MSFT price trajectory.

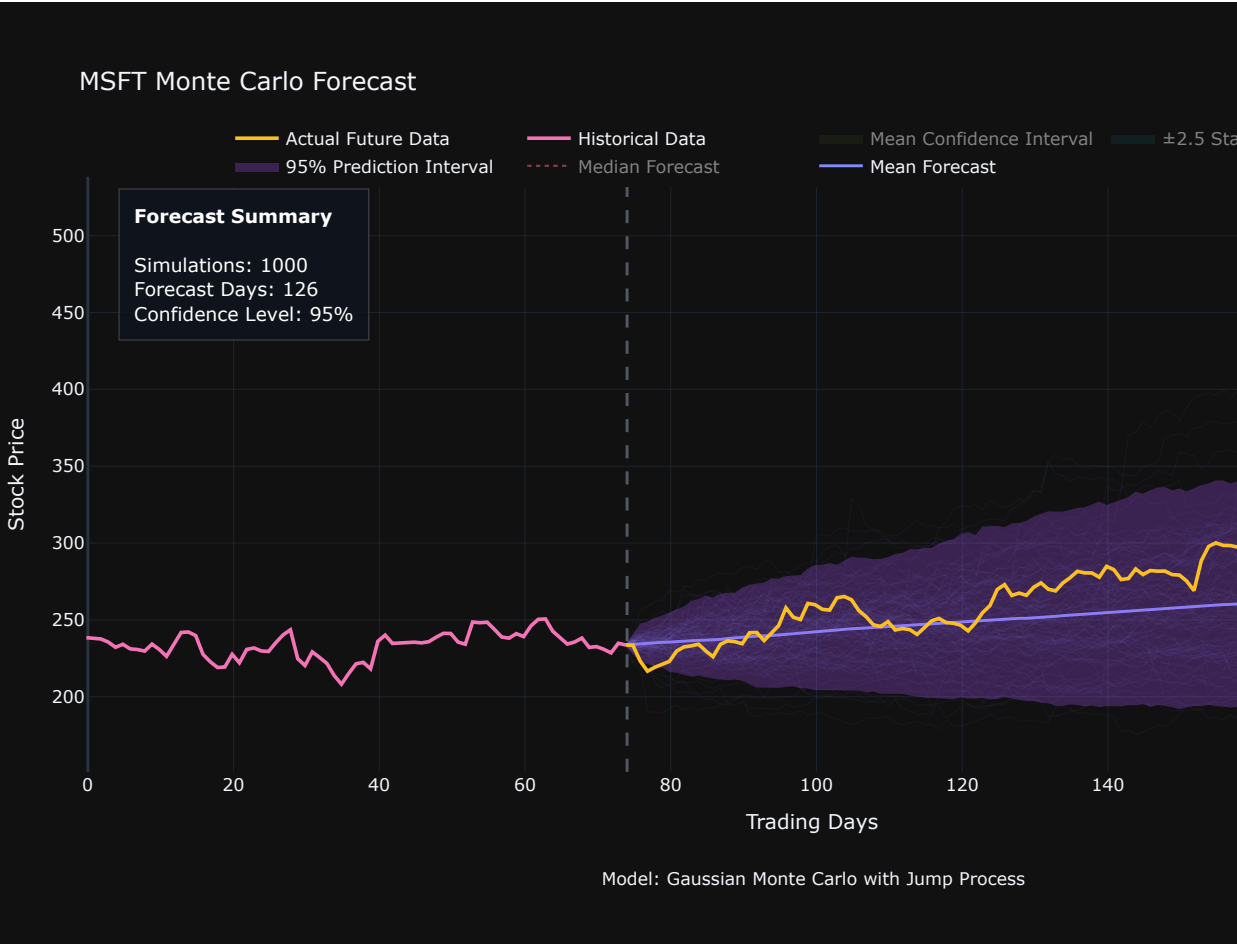
```

In [11]: plot_results(
    simulations = montecarlo_data[stock_name[2]][1],
    confidence_level = confidence_level,
    sd_multiplier=sigma_multiplier,
    stock_name = stock_name[2],
    historical_data = training_stock_data[stock_name[2]]["Close"],
    actual_data = evaluation_stock_data[stock_name[2]]["Close"].iloc[0:montecarlo_days-1],
    allow_jumps = True
).show()

```

```
#the remaining parameters can be the default since plotly allows for interactive graphs

#the evaluation parameters to be shown.
show_evaluation(
  simulations = montecarlo_data[stock_name[2]][1],
  actual_data = evaluation_stock_data[stock_name[2]]["Close"],
  training_data = training_stock_data[stock_name[2]]["Close"],
  stock_name = stock_name[2],
  jumps_enabled=True,
  confidence_level=confidence_level
)
```



=====			
MSFT With Jumps			
=====			
RMSE	34.0923	MAPE (%)	9.0204
Coverage (%)	96.83	Dispersion	1.531555

	Real	Simulation	
Positive Jumps	5	1.99	
Negative Jumps	1	2.09	
Avg Positive Jump	0.0481	0.0648	
Avg Negative Jump	-0.0437	-0.0571	

Introducing jump dynamics increased forecast dispersion and improved prediction interval coverage from 91.27% to 96.83%, indicating that the broader uncertainty estimates were better able to contain the real market movements. However, this improvement came alongside slightly worse RMSE and MAPE values. This suggests that while the jump diffusion framework produced a more realistic distribution of possible outcomes, the average simulated trajectory moved further away from the real price path. One likely explanation is that the historical calibration process treated positive and negative jumps more symmetrically, whereas the validation period for Microsoft was dominated primarily by positive jump behaviour associated with strong recent performance in the technology sector. As a result, the model successfully captured increased volatility and uncertainty, but did not fully reproduce the directional bias of the observed upward market movements.

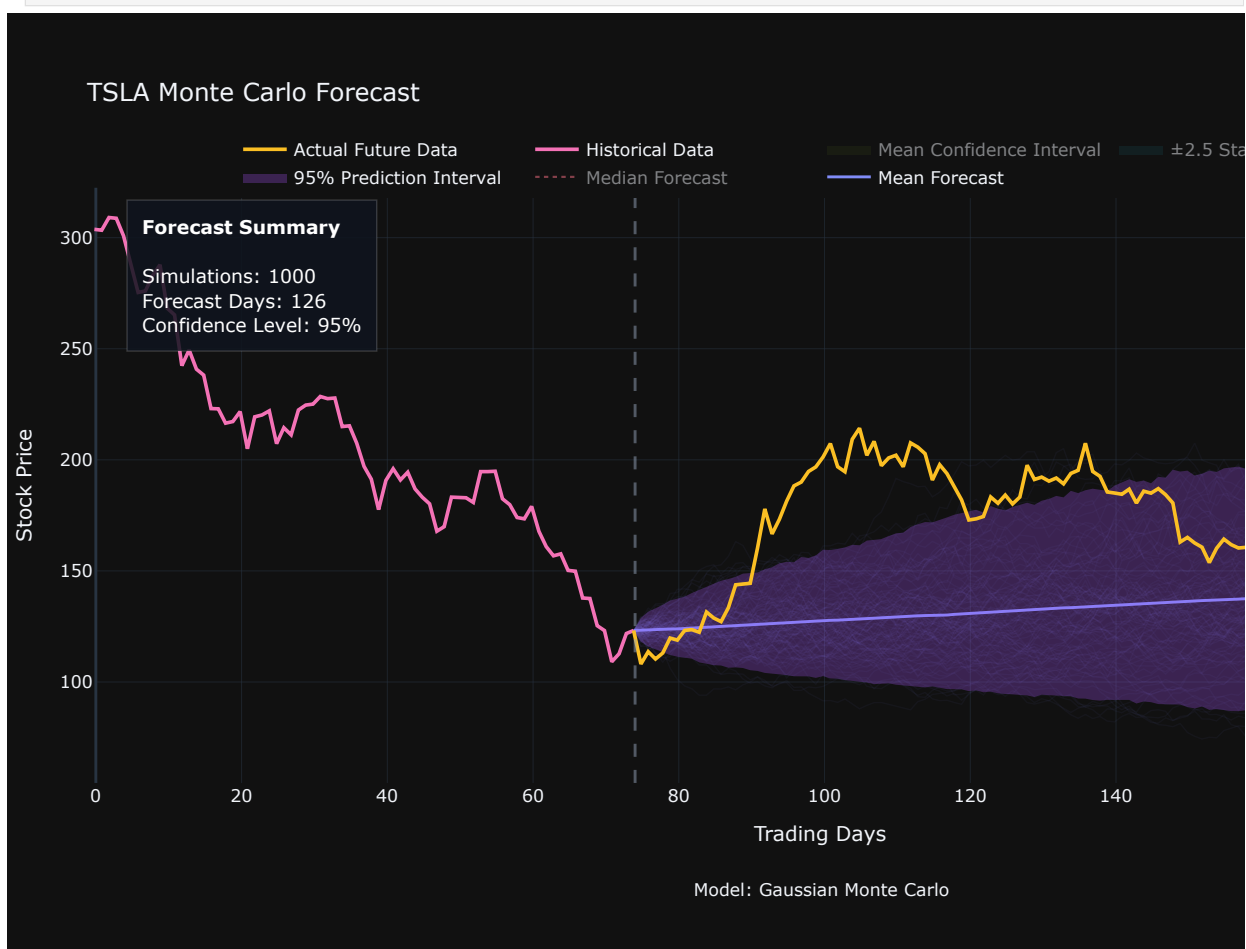
TSLA (Tesla)

High-volatility growth stock heavily influenced by sentiment, innovation cycles, and macroeconomic expectations. Included to test the model under extreme price fluctuations and regime shifts.

```
In [12]: plot_results(
    simulations = montecarlo_data[stock_name[3]][0],
    confidence_level = confidence_level,
    sd_multiplier=sigma_multiplier,
    stock_name = stock_name[3],
    historical_data = training_stock_data[stock_name[3]]["Close"],
    actual_data = evaluation_stock_data[stock_name[3]]["Close"].iloc[0:montecarlo_days-1],
    allow_jumps = False
).show()

#the remaining parameters can be the default since plotly allows for interactive graphs

#the evaluation parameters to be shown.
show_evaluation(
    simulations = montecarlo_data[stock_name[3]][0],
    actual_data = evaluation_stock_data[stock_name[3]]["Close"],
    training_data = training_stock_data[stock_name[3]]["Close"],
    stock_name = stock_name[3],
    jumps_enabled = False,
    confidence_level=confidence_level
)
```



=====

TSLA | Standard Model

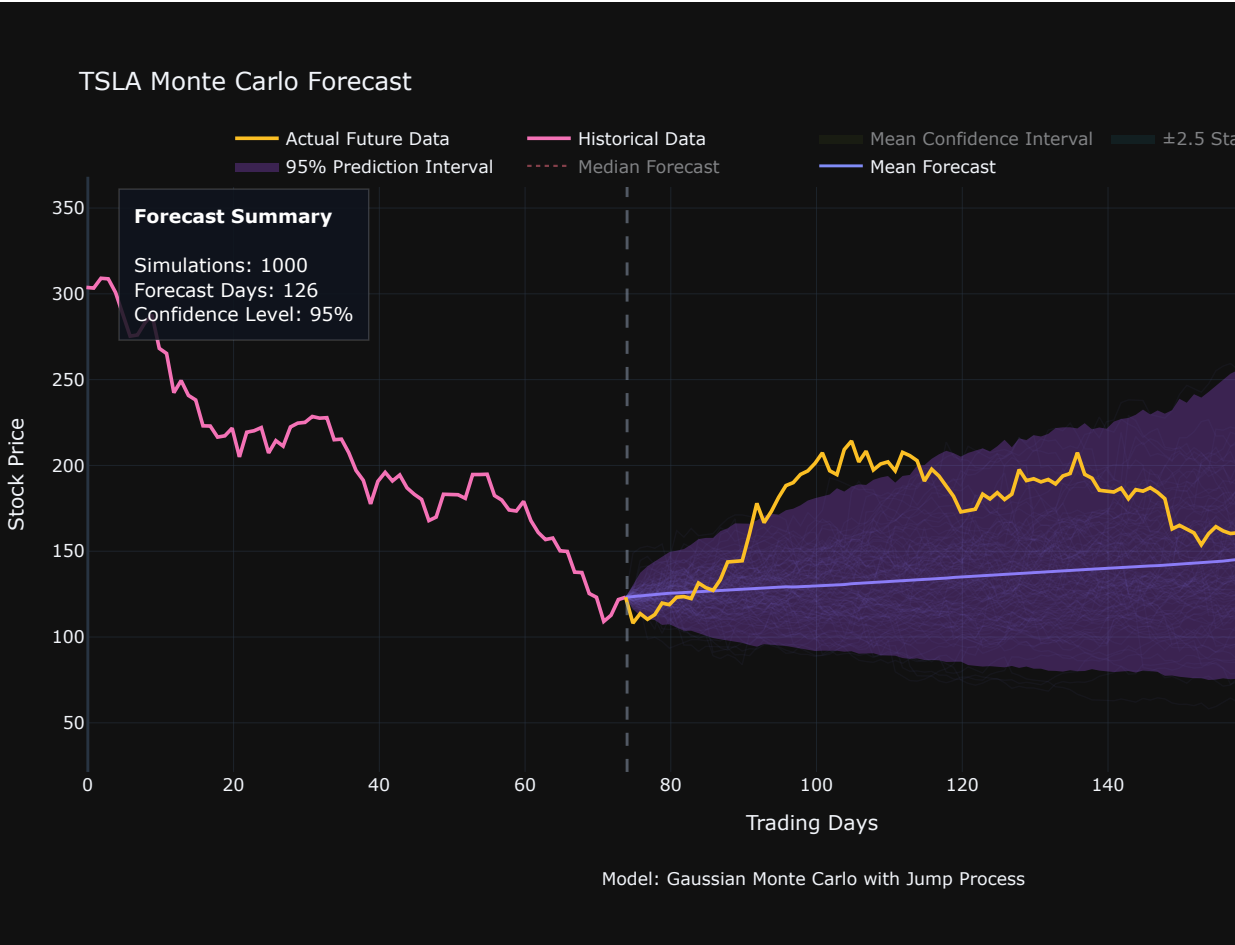
=====

RMSE	64.1737	MAPE (%)	27.3079
Coverage (%)	43.65	Dispersion	1.141606

	Real	Simulation
Positive Jumps	6	0.08
Negative Jumps	1	0.10
Avg Positive Jump	0.0874	0.0761
Avg Negative Jump	-0.0975	-0.0730

Tesla represents the clearest example within these case studies of a stock requiring jump diffusion dynamics in order to produce realistic simulations. The real TSLA price trajectory contained a large number of significant discontinuous movements, particularly strong positive jumps, which our standard framework struggled to reproduce. As a result, the standard model produced very weak prediction interval coverage of only 43.65%, indicating that the simulated uncertainty bands were far too narrow for the observed market behaviour. The relatively low (essentially 0) simulated jump counts generated by the standard model further demonstrate its inability to capture the heavy-tailed and highly volatile nature of Tesla's price dynamics. Even without explicit jumps, the forecast dispersion remained noticeably larger than for more stable equities, reflecting the inherently high volatility of the stock.

```
In [13]: plot_results(  
    simulations = montecarlo_data[stock_name[3]][1],  
    confidence_level = confidence_level,  
    sd_multiplier=sigma_multiplier,  
    stock_name = stock_name[3],  
    historical_data = training_stock_data[stock_name[3]]["Close"],  
    actual_data = evaluation_stock_data[stock_name[3]]["Close"].iloc[0:montecarlo_days-1],  
    allow_jumps = True  
).show()  
  
#the remaining parameters can be the default since plotly allows for interactive graphs  
  
#the evaluation parameters to be shown.  
show_evaluation(  
    simulations = montecarlo_data[stock_name[3]][1],  
    actual_data = evaluation_stock_data[stock_name[3]]["Close"],  
    training_data = training_stock_data[stock_name[3]]["Close"],  
    stock_name = stock_name[3],  
    jumps_enabled=True,  
    confidence_level=confidence_level  
)
```



=====			
TSLA With Jumps			
=====			
RMSE	57.5651	MAPE (%)	24.2570
Coverage (%)	80.16	Dispersion	1.907613

	Real	Simulation	
Positive Jumps	6	2.65	
Negative Jumps	1	2.01	
Avg Positive Jump	0.0874	0.1220	
Avg Negative Jump	-0.0975	-0.1144	

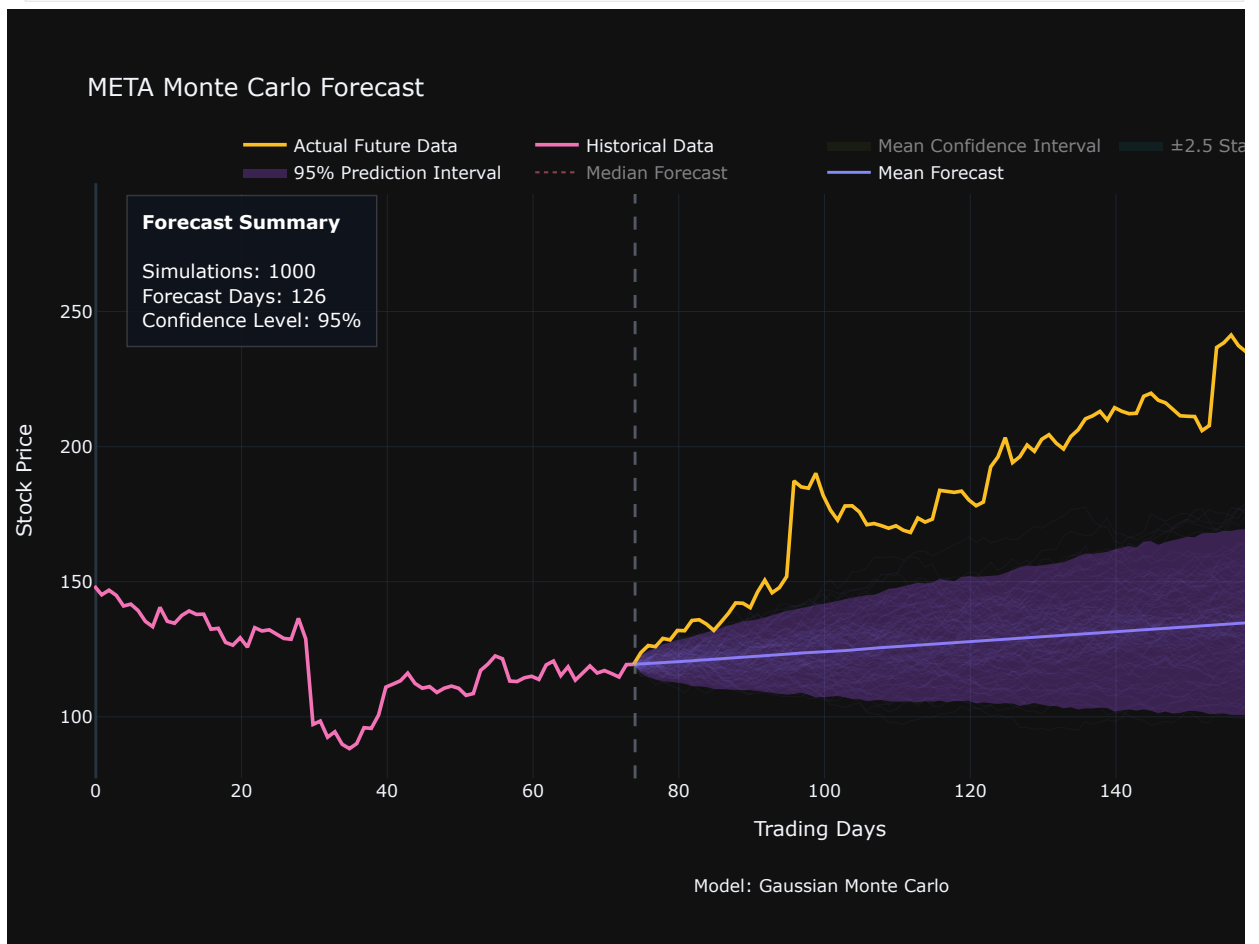
Introducing jump dynamics produced substantial improvements across nearly all evaluation metrics. RMSE and MAPE both decreased, while prediction interval coverage improved significantly from 44.44% to 80.16%. The jump framework also generated much wider confidence intervals, producing a dispersion profile that more realistically reflected the uncertainty associated with Tesla's highly volatile price behaviour. Unlike previous case studies where jump frequency was often overestimated, the TSLA simulations still underestimated the number of real jumps observed during the validation period. This suggests that the calibration procedure, while improved through numerical optimisation, may still apply jump thresholds that are too conservative for extremely high-volatility assets. The validation period also contained predominantly positive jump behaviour, whereas the model generated a more symmetric balance between positive and negative jumps based on historical distributions. Nevertheless, the jump model

clearly provided a substantially more realistic representation of Tesla's market dynamics and demonstrates the importance of incorporating large price movements when modelling highly volatile growth equities.

META (Meta/Facebook)

Momentum-driven technology company operating in social media and digital advertising. Included due to its strong directional movements and periods of rapid repricing.

```
In [14]: plot_results(  
    simulations = montecarlo_data[stock_name[4]][0],  
    confidence_level = confidence_level,  
    sd_multiplier=sigma_multiplier,  
    stock_name = stock_name[4],  
    historical_data = training_stock_data[stock_name[4]]["Close"],  
    actual_data = evaluation_stock_data[stock_name[4]]["Close"].iloc[0:montecarlo_days-1],  
    allow_jumps = False  
).show()  
  
#the remaining parameters can be the default since plotly allows for interactive graphs  
  
#the evaluation parameters to be shown.  
show_evaluation(  
    simulations = montecarlo_data[stock_name[4]][0],  
    actual_data = evaluation_stock_data[stock_name[4]]["Close"],  
    training_data = training_stock_data[stock_name[4]]["Close"],  
    stock_name = stock_name[4],  
    jumps_enabled=False,  
    confidence_level=confidence_level  
)
```

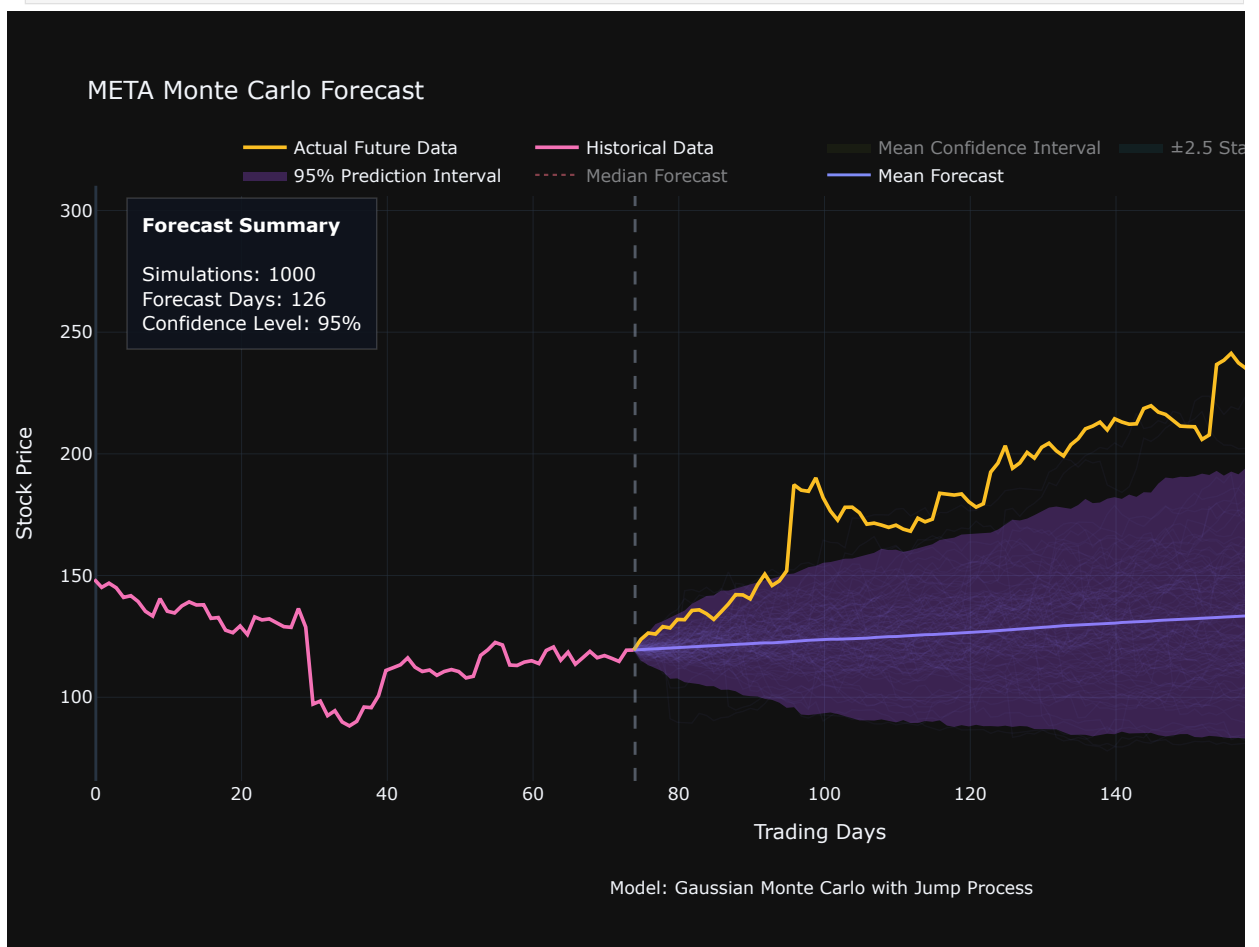


META | Standard Model

RMSE	86.7538	MAPE (%)	34.4008
Coverage (%)	0.00	Dispersion	0.703632

	Real	Simulation
Positive Jumps	4	0.04
Negative Jumps	0	0.01
Avg Positive Jump	0.1265	0.0483
Avg Negative Jump	0.0000	-0.0468

```
In [15]: plot_results(  
    simulations = montecarlo_data[stock_name[4]][1],  
    confidence_level = confidence_level,  
    sd_multiplier=sigma_multiplier,  
    stock_name = stock_name[4],  
    historical_data = training_stock_data[stock_name[4]]["Close"],  
    actual_data = evaluation_stock_data[stock_name[4]]["Close"].iloc[0:montecarlo_days-1],  
    allow_jumps = True  
).show()  
  
#the remaining parameters can be the default since plotly allows for interactive graphs  
  
#the evaluation parameters to be shown.  
show_evaluation(  
    simulations = montecarlo_data[stock_name[4]][1],  
    actual_data = evaluation_stock_data[stock_name[4]]["Close"],  
    training_data = training_stock_data[stock_name[4]]["Close"],  
    stock_name = stock_name[4],  
    jumps_enabled=True,  
    confidence_level=confidence_level  
)
```



META | With Jumps

RMSE	88.1285	MAPE (%)	34.8793
Coverage (%)	12.70	Dispersion	1.152513

	Real	Simulation
Positive Jumps	4	2.25
Negative Jumps	0	2.19
Avg Positive Jump	0.1265	0.0796
Avg Negative Jump	0.0000	-0.0841

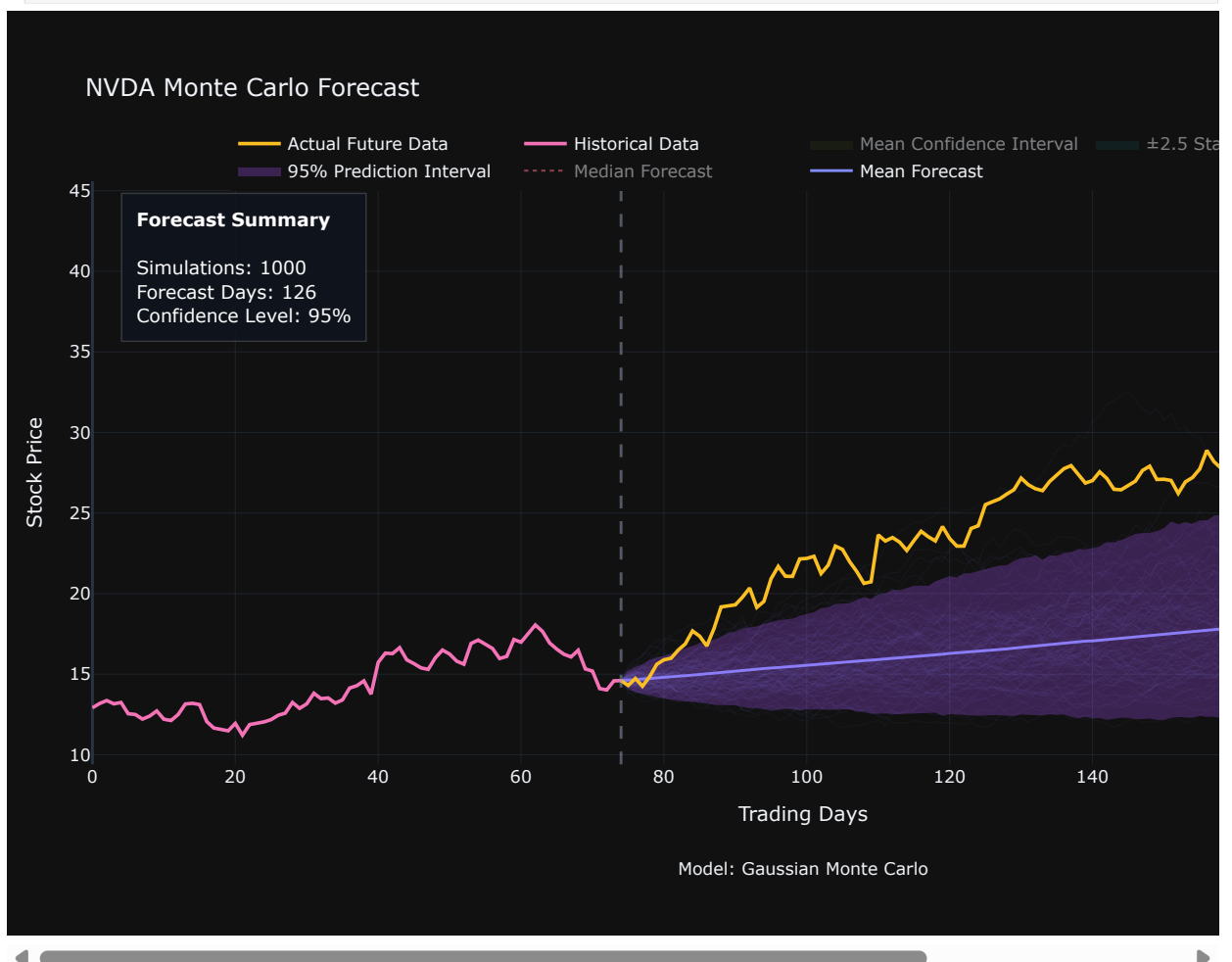
META represents the clearest failure case within this study and highlights one of the fundamental limitations of historically calibrated Monte Carlo forecasting models. Both our standard and jump frameworks performed extremely poorly, with very large RMSE and MAPE values alongside exceptionally weak prediction interval coverage. Even after incorporating jump dynamics, coverage improved only marginally from 0% to 12.7%, indicating that the real price trajectory consistently evolved outside the range of outcomes predicted by the model. While the jump extension increased forecast dispersion and generated more discontinuous movements, the simulated jump behaviour still failed to reproduce the magnitude and directional persistence of the actual market movement observed during the validation period.

A likely explanation for this behaviour is that META experienced a substantial structural or regime shift beginning around early 2023, during which the stock underwent a sustained upward revaluation that was not well represented within the historical training data. Historically, the calibrated jump process assumed a more balanced distribution between positive and negative discontinuous movements. However, the real validation period was dominated almost entirely by strong positive jumps and persistent upward momentum. As a result, the model systematically underestimated both the frequency and magnitude of positive market movements. Although a small number of individual Monte Carlo paths did reach price levels similar to the real trajectory, these simulations existed within the extreme tail region outside the primary confidence interval and therefore did not represent the model's central forecast expectation. This case study demonstrates that even jump extensions may fail when future market behaviour is driven by structural changes, regime transitions, or prolonged directional momentum that differs substantially from historical dynamics.

NVDA (NVIDIA)

Semiconductor and AI infrastructure company associated with exceptionally strong momentum and rapid valuation growth during the AI era. Included to evaluate model behaviour under sustained high-growth conditions.

```
In [16]: plot_results(  
    simulations = montecarlo_data[stock_name[5]][0],  
    confidence_level = confidence_level,  
    sd_multiplier=sigma_multiplier,  
    stock_name = stock_name[5],  
    historical_data = training_stock_data[stock_name[5]]["Close"],  
    actual_data = evaluation_stock_data[stock_name[5]]["Close"].iloc[0:montecarlo_days-1],  
    allow_jumps = False  
).show()  
  
#the remaining parameters can be the default since plotly allows for interactive graphs  
  
#the evaluation parameters to be shown.  
show_evaluation(  
    simulations = montecarlo_data[stock_name[5]][0],  
    actual_data = evaluation_stock_data[stock_name[5]]["Close"],  
    training_data = training_stock_data[stock_name[5]]["Close"],  
    stock_name = stock_name[5],  
    jumps_enabled=False,  
    confidence_level=confidence_level  
)
```

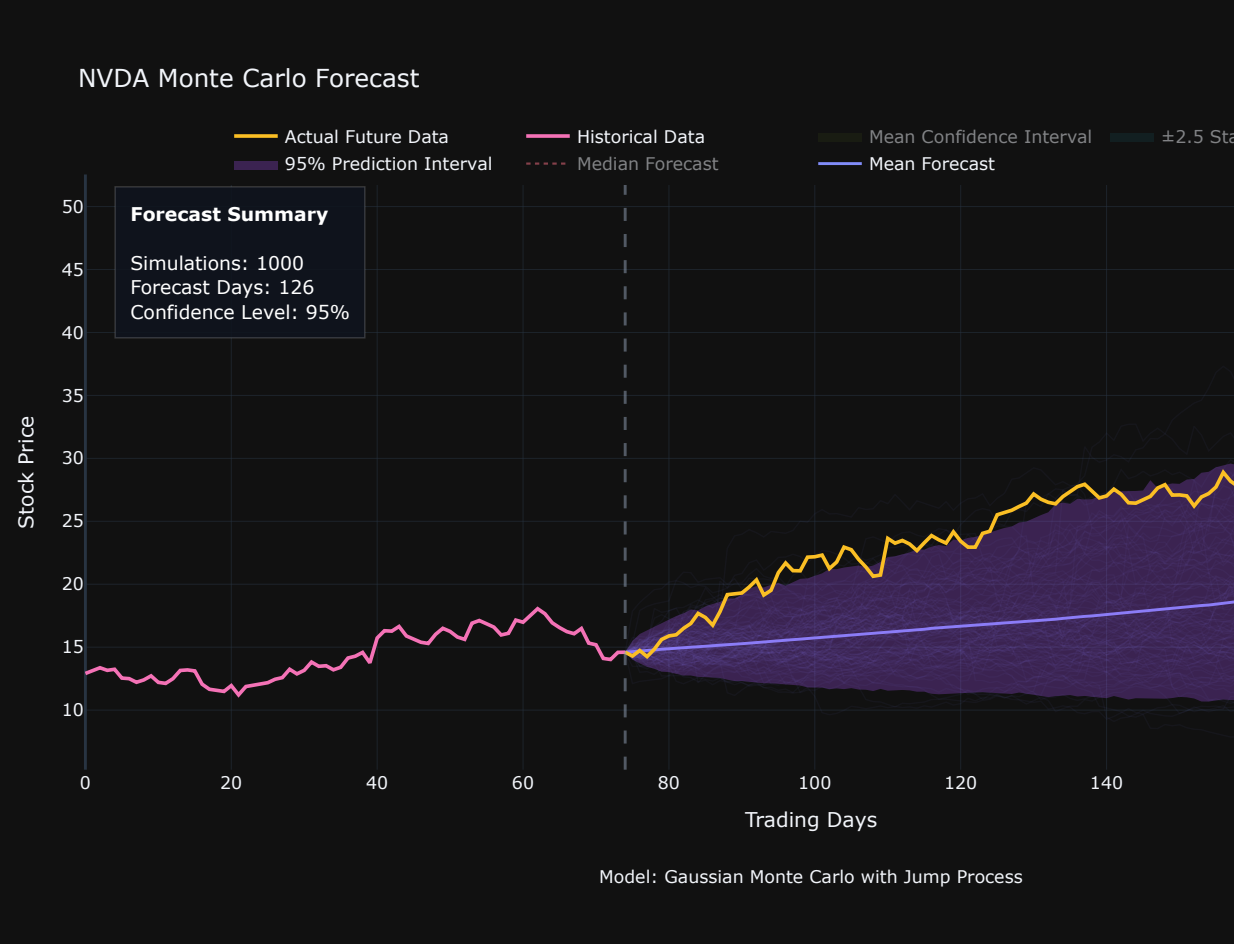


NVDA Standard Model			
RMSE	12.4922	MAPE (%)	35.0294
Coverage (%)	5.56	Dispersion	0.136252
	Real	Simulation	
Positive Jumps	5	0.09	
Negative Jumps	1	0.08	
Avg Positive Jump	0.1192	0.0661	
Avg Negative Jump	-0.0591	-0.0611	

```
In [17]: plot_results(
    simulations = montecarlo_data[stock_name[5]][1],
    confidence_level = confidence_level,
    sd_multiplier=sigma_multiplier,
    stock_name = stock_name[5],
    historical_data = training_stock_data[stock_name[5]]["Close"],
    actual_data = evaluation_stock_data[stock_name[5]]["Close"].iloc[0:montecarlo_days-1],
    allow_jumps = True
).show()

#the remaining parameters can be the default since plotly allows for interactive graphs

#the evaluation parameters to be shown.
show_evaluation(
    simulations = montecarlo_data[stock_name[5]][1],
    actual_data = evaluation_stock_data[stock_name[5]]["Close"],
    training_data = training_stock_data[stock_name[5]]["Close"],
    stock_name = stock_name[5],
    jumps_enabled=True,
    confidence_level=confidence_level
)
```



=====			
NVDA With Jumps			
=====			
RMSE	11.8388	MAPE (%)	33.2671
Coverage (%)	39.68	Dispersion	0.214152

	Real	Simulation	
Positive Jumps	5	2.21	
Negative Jumps	1	1.88	
Avg Positive Jump	0.1192	0.1077	
Avg Negative Jump	-0.0591	-0.0926	

NVIDIA represents a similar case to META, although the jump framework performed somewhat better due to the model partially capturing the early stages of the AI-driven growth period within the historical training data. Our standard model severely underestimated the frequency of large movements, resulting in extremely poor prediction interval coverage and insufficient forecast dispersion. Introducing jump diffusion dynamics improved RMSE, MAPE, and interval coverage, while also producing simulated jump magnitudes closer to those observed in the real market data.

However, both models still struggled to reproduce the sustained upward momentum present throughout the validation period. The real stock behaviour was dominated primarily by large positive jumps associated with the rapid expansion of the AI sector, whereas the simulation continued to generate a more balanced mixture of positive and negative based on historical date predating the modern AI sector. This suggests that although the historical calibration process captured part of the emerging AI-related growth trend, it still underestimated the degree of positive asymmetry and directional persistence present within NVIDIA’s real market trajectory.

Conclusions and next Steps

Overall, the Monte Carlo framework developed throughout this project demonstrated strong performance when modelling relatively stable equities and broad market indices. For stocks such as SPY and Coca-Cola, the model was generally capable of reproducing realistic market behaviour, particularly when jump dynamics were incorporated where necessary. Stable equities naturally exhibited lower forecast dispersion and fewer discontinuous movements, which aligned well with the smoother behaviour observed within the real market data. In particular, the framework performed very well for SPY, where large discontinuous movements were largely absent throughout the validation period. This supports the broader financial intuition that diversified market indices tend to smooth out extreme idiosyncratic movements from individual companies.

The addition of jumps became increasingly important as stock volatility increased. In more volatile equities such as Tesla, the standard framework struggled to capture the frequency and magnitude of discontinuous price movements, leading to poor confidence interval coverage and weaker forecast accuracy. Incorporating jumps substantially improved nearly all evaluation metrics and produced uncertainty bands that more realistically reflected the highly volatile nature of the stock. This demonstrates that explicit jump modelling is not only beneficial but often necessary when simulating highly volatile growth equities. More generally, the results throughout the case studies showed that jumps typically improved prediction interval coverage and uncertainty estimation, even in situations where point forecast metrics such as RMSE or MAPE did not improve significantly.

At the same time, the project also clearly demonstrated the limitations of historically calibrated Monte Carlo forecasting frameworks. The model struggled substantially during periods of structural market change or regime shifts that were not represented within the historical training data. META provided the clearest example of this behaviour, where a major restructuring and reassessment by investors during early 2023 resulted in an extreme and sustained upward movement that the simulation failed to reproduce. NVIDIA exhibited a similar pattern during the rapid AI-driven market expansion. Although the training data partially captured the early stages of this trend and therefore introduced some positive asymmetry into the simulated jump behaviour, the framework still underestimated both the magnitude and persistence of the real upward movement. These case studies highlight an important limitation of purely historical calibration: future market behaviour may differ substantially from past dynamics during periods of structural change, sector expansion, or strong directional momentum.

The results also suggest that the current jump calibration procedure, while already incorporating historical jump behaviour, may still apply optimisation too uniformly across different classes of equities. In several stable stocks, the framework appeared to slightly overestimate jump frequency and jump dispersion, whereas highly volatile equities such as Tesla still exhibited fewer jumps and less extreme movement than were observed within the real market data. This suggests that the current calibration process behaves somewhat like a global average across volatility regimes rather than fully adapting to the characteristics of individual equities.

One potential improvement would therefore be the introduction of a more volatility-sensitive jump calibration framework, where jump thresholds and jump frequencies are adjusted according to the historical volatility profile of the underlying asset. Stable equities would therefore generate fewer simulated jumps and narrower uncertainty distributions than are currently produced by the framework, while highly volatile growth equities would allow for more aggressive jump behaviour and wider confidence intervals. Although this would likely increase confidence interval growth and reduce forecast precision for unstable equities, it may simultaneously improve overall forecast accuracy and realism.

Another possible improvement would involve introducing weighted historical training data, allowing more recent market behaviour to exert greater influence over the simulation process than older observations. Financial markets often display trend persistence and

evolving structural behaviour, meaning recent data may be more representative of near-future conditions than long-term historical averages. Such an approach may have improved performance in cases such as NVIDIA, where the AI-driven growth trend had already begun to emerge within the later sections of the training data. More advanced extensions could also sequentially update simulation behaviour using previously generated daily movements, allowing the framework to dynamically adapt as trends develop throughout the forecast horizon. However, this would substantially increase computational and modelling complexity.

Further improvements may also be achieved by relaxing the Gaussian assumptions underlying the simulation framework. Real financial markets frequently exhibit heavy-tailed return distributions, asymmetric jump behaviour, and clustering of volatility that are difficult to reproduce using standard Gaussian processes alone. Alternative return distributions or stochastic volatility approaches may therefore provide more realistic simulations, particularly during periods of market stress. Finally, incorporating broader market or sector-level behaviour into the framework could improve predictive realism further, as equities within the same industry often display correlated movement patterns and respond similarly to wider macroeconomic conditions. Despite these limitations, the framework developed throughout this project successfully demonstrated the strengths and weaknesses of Monte Carlo stock price simulation across a diverse range of market behaviours, volatility profiles, and structural market conditions.